

Submission Worksheet

Submission Data

Course: IT114-450-M2025

Assignment: IT114 - Milestone 3 - Trivia

Student: Preet S. (ps55)

Status: Submitted | **Worksheet Progress:** 76%

Potential Grade: 6.96/10.00 (69.60%)

Received Grade: 0.00/10.00 (0.00%)

Started: 8/11/2025 9:57:05 PM

Updated: 8/11/2025 11:38:08 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT114-450-M2025/it114-milestone-3-trivia/grading/ps55>

View Link: <https://learn.ethereallab.app/assignment/v3/IT114-450-M2025/it114-milestone-3-trivia/view/ps55>

Instructions

1. Refer to Milestone3 of [Trivia](#)
 1. Complete the features
2. Ensure all code snippets include your ucid, date, and a brief description of what the code does
3. Switch to the Milestone3 branch
 1. `git checkout Milestone3`
 2. `git pull origin Milestone3`
4. Fill out the below worksheet as you test/demo with 3+ clients in the same session
5. Once finished, click "Submit and Export"
6. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 1. `git add .`
 2. `git commit -m "adding PDF"`
 3. `git push origin Milestone3`
 4. On Github merge the pull request from Milestone3 to main
7. Upload the same PDF to Canvas
8. Sync Local
 1. `git checkout main`
 2. `git pull origin main`

Section #1: (1 pt.) Core Ui

Progress: 87%

☰ Task #1 (0.50 pts.) - Connection/Details Panels

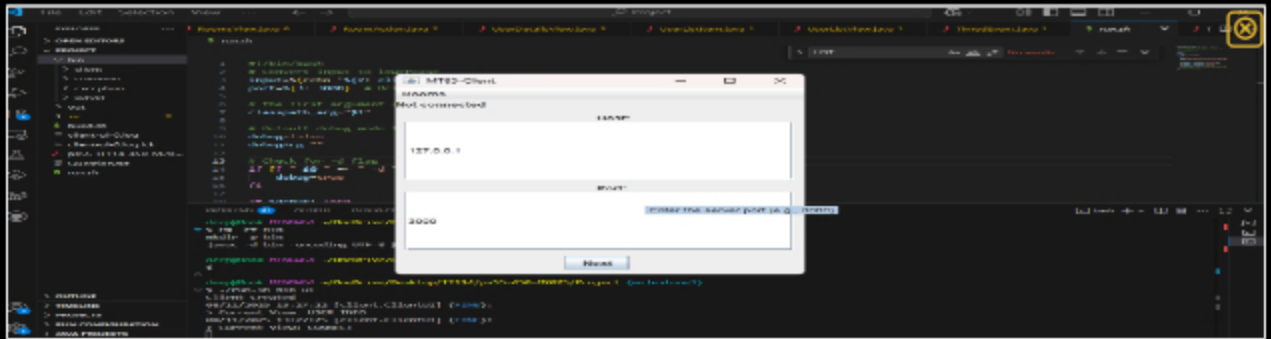
Progress: 100%

📄 Part 1:

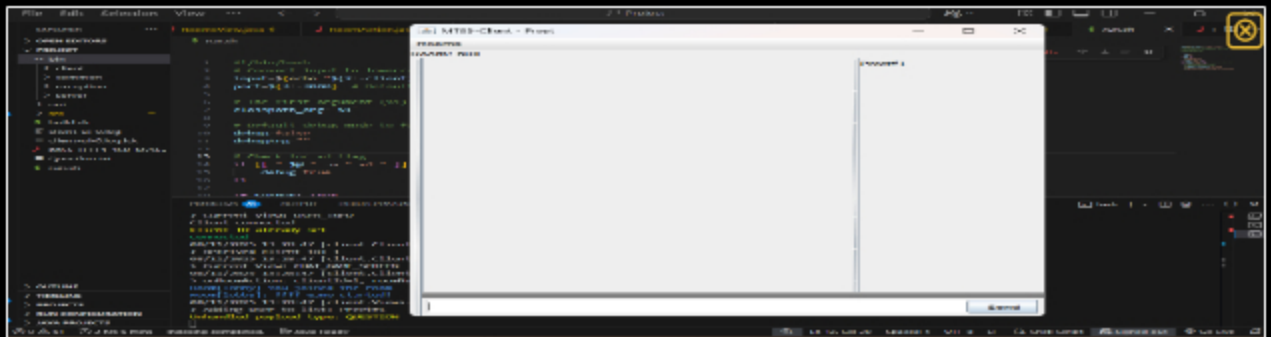
Progress: 100%

Details:

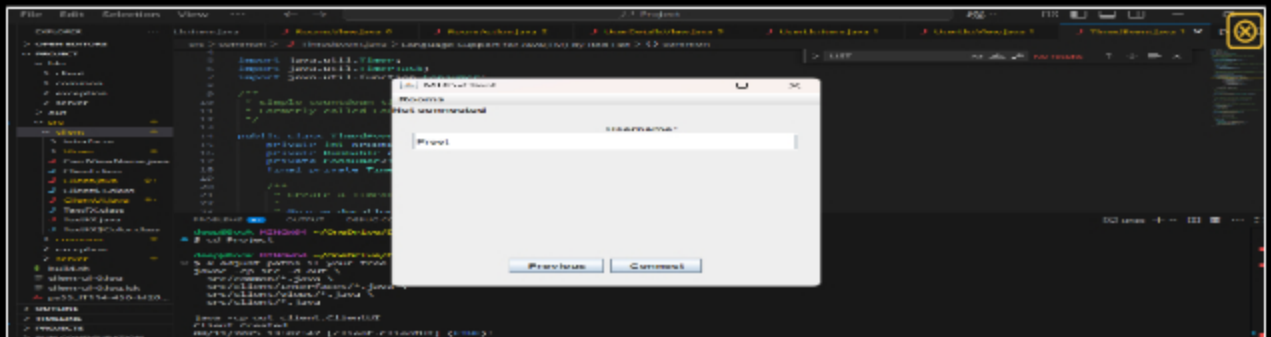
- Show the connection panel with valid data
- Show the user details panel with valid data



Connection



Room



Username



Saved: 8/11/2025 10:28:46 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow from recording/capturing these details and passing them through the connection process

Your Response:

The username is entered in UserDetailsView and the host/port in ConnectionView. When the user

clicks Connect, ClientUI.connect() retrieves these values and calls Client.connect(), which opens a socket to the server and sends a CLIENT_CONNECT payload with the username. The server registers the client, sends back a client ID, and ClientUI switches the UI to the main game/chat screen

 Saved: 8/11/2025 10:28:46 PM

≡ Task #2 (0.50 pts.) - Ready Panel

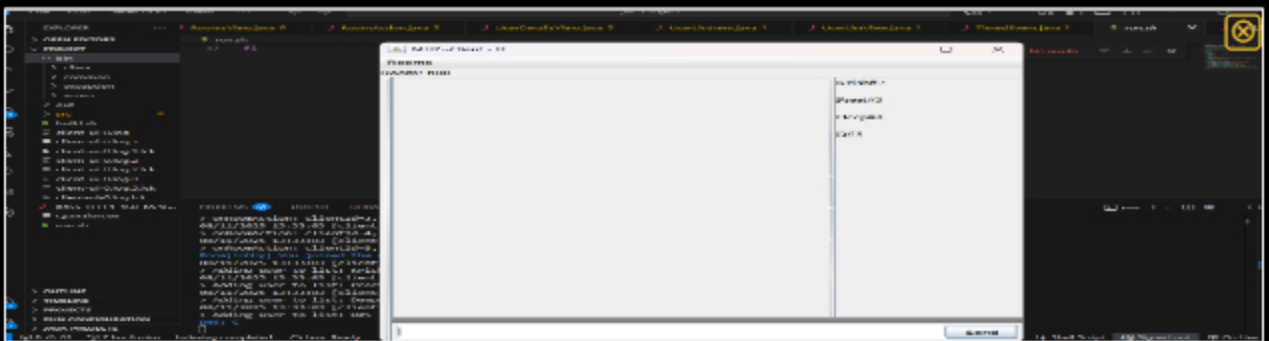
Progress: 75%

Part 1:

Progress: 50%

Details:

- Show the button used to mark ready
- Show a few variations of indicators of clients being ready (3+ clients)



3+Clients



Missing Caption

 Saved: 8/11/2025 10:31:27 PM

⇒ Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for marking READY from the UI

- Briefly explain the code flow from receiving READY data and updating the UI

Your Response:

Marking READY from the UI

When a player clicks the Ready button in the ReadyView panel, the button's ActionListener triggers the Client.INSTANCE.sendReady() method. This method creates a Payload with the type READY and sends it to the server through the client's socket connection. The server receives the payload, updates the ready status for that player in the active room, and then broadcasts a synchronization payload (SYNC_READY) to all connected clients in that room.

Receiving READY Data and Updating the UI

On each client, the network handling thread receives the SYNC_READY payload from the server. The Client instance routes this data to the appropriate callback method (for example, in a UserListView or lobby UI controller). This callback updates the internal representation of each player's ready state. Finally, the UI is refreshed to display the updated ready indicators for all players — this can be visual cues such as green checkmarks, highlighted rows, or icons next to usernames.



Saved: 8/11/2025 10:31:27 PM

Section #2: (2 pts.) Project Ui

Progress: 83%

≡ Task #1 (0.67 pts.) - User List Panel

Progress: 100%

Details:

- Show the username and id of each user
- Show the current points of each user
- Users should appear in same order across each client
- Show an indicator of who locked in their answer

🖼 Part 1:

Progress: 100%

Details:

- Show various examples of points (3+ clients visible)
 - Include code snippets showing the code flow for this from server-side to UI
- Show that the sorting is maintained across clients
 - Include code snippets showing the code that handles this
- Show various examples of locked-in indicator
 - Include code snippets showing the code flow for this from server-side to UI


```
broadcastToRoom(roomName, p);
```

Sortedroom

```
case SYNC_CLIENT: {  
    List<User> received = payload.getUsers();  
    userListView.setUsersInOrder(received);  
    break;  
}
```

SortedClient

```
case SYNC_TURN: {  
    long id = payload.getClientId();  
    String msg = payload.getMessage(); // e.g., "LOCKED_IN:TRUE;CORRECT:FALSE"  
    boolean locked = msg.contains("LOCKED_IN:TRUE");  
    boolean correct = msg.contains("CORRECT:TRUE");  
    userListView.markLockedIn(id, locked, correct);  
    break;  
}  
case RESET_TURN: {  
    userListView.clearLockedInAll();  
    break;  
}
```

Locked-Client

```
public void addRoom(Room r) {  
    // Add room to the list of rooms  
    rooms.add(r);  
    // Add room to the list of rooms  
    rooms.add(r);  
}  
  
public void addClient(Client c) {  
    // Add client to the list of clients  
    clients.add(c);  
    // Add client to the list of clients  
    clients.add(c);  
}  
  
public void addMessage(Message m) {  
    // Add message to the list of messages  
    messages.add(m);  
    // Add message to the list of messages  
    messages.add(m);  
}  
  
public void addLockedIn(long clientId, boolean lockedIn) {  
    // Add locked-in status to the list of locked-in clients  
    lockedInClients.add(clientId, lockedIn);  
    // Add locked-in status to the list of locked-in clients  
    lockedInClients.add(clientId, lockedIn);  
}  
  
public void addCorrect(long clientId, boolean correct) {  
    // Add correct status to the list of correct clients  
    correctClients.add(clientId, correct);  
    // Add correct status to the list of correct clients  
    correctClients.add(clientId, correct);  
}
```

LockedUserListView



Saved: 8/11/2025 11:00:51 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for points updates from server-side to the UI
- Briefly explain the code flow for user list sorting
- Briefly explain the code flow for server-side to UI of locked-in indicators

Your Response:

1. Points updates When someone scores points in the game, the server updates their score and sends that new points info to everyone. The client receives it, passes it to the UI, and the player's score on the screen changes right away.
2. User list sorting The server keeps the list of players in a fixed order, usually based on when they joined or by their ID. It sends that list to all clients in the same order, so everyone sees the players arranged the same way.
3. Locked-in indicators When a player locks in their answer, the server marks that in its records and tells all the clients. The clients then update the display, maybe showing a checkmark, greying out the name, or adding some other icon, so everyone knows that player is locked in. When a new round starts, the server clears those indicators.



Saved: 8/11/2025 11:00:51 PM

≡ Task #2 (0.67 pts.) - Game Events Panel

Progress: 100%

Details:

- Show the answer lock-in history
 - (i.e., Bob locked in the correct answer or Bob locked in the wrong answer)
- Show earned points
- Show the scoreboard updates from Milestone2

🖼 Part 1:

Progress: 100%

Details:

- Show various examples of each of the messages/visuals
- Show code snippets related to these messages from server-side to UI
- Show the countdown timer for the round

```
void tickRoundTimer(String roomName, int secondsLeft) {  
    Payload t = new Payload();  
    t.setPayloadType(PayloadType.TIME);  
    t.setMessage("ROUND:" + secondsLeft);  
    broadcastToRoom(roomName, t);  
}
```

Room

```
void handleAnswer(String roomName, long clientId, int choiceIndex) {  
    boolean correct = isCorrect(choiceIndex);  
    lockedIn.put(clientId, Boolean.TRUE);  
    if (correct) {  
        // ...  
    }  
}
```

```

    int newPt = point.getAddress(clientId, area);
    Payload pts = new Payload();
    pts.setPayloadType(PayloadType.POINTS);
    pts.setClientId(clientId);
    pts.setMessage(String.valueOf(newPt));
    broadcastToRoom(roomName, pts);
}

Payload sync = new Payload();
sync.setPayloadType(PayloadType.SYNC_TURN);
sync.setClientId(clientId);
sync.setMessage(correct ? "LOCKED_IN:TRUE;CORRECT:TRUE" : "LOCKED_IN:TRUE;CORRECT:FALSE");
broadcastToRoom(roomName, sync);
Payload cvl = new Payload();
cvl.setPayloadType(PayloadType.MESSAGE);
cvl.setClientId(constants.DEFAULT_CLIENT_ID);
cvl.setMessage(getDisplayName(clientId) + (correct ? " locked in the correct answer" : " locked in the wrong answer"));
broadcastToRoom(roomName, cvl);
}

```

Room

```

@Override
public void handleMessage() {
    Payload payload = getPayload();
    if (payload == null) {
        return;
    }
    if (payload.getPayloadType() == PayloadType.POINTS) {
        int newPt = payload.getAddress(clientId, area);
        pts.setClientId(clientId);
        pts.setMessage(String.valueOf(newPt));
        broadcastToRoom(roomName, pts);
    }
    if (payload.getPayloadType() == PayloadType.SYNC_TURN) {
        boolean correct = payload.getMessage().contains("CORRECT:TRUE");
        sync.setClientId(clientId);
        sync.setMessage(correct ? "LOCKED_IN:TRUE;CORRECT:TRUE" : "LOCKED_IN:TRUE;CORRECT:FALSE");
        broadcastToRoom(roomName, sync);
    }
    if (payload.getPayloadType() == PayloadType.MESSAGE) {
        cvl.setClientId(constants.DEFAULT_CLIENT_ID);
        cvl.setMessage(getDisplayName(clientId) + (correct ? " locked in the correct answer" : " locked in the wrong answer"));
        broadcastToRoom(roomName, cvl);
    }
}

```

Client



Saved: 8/11/2025 11:12:38 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for generating these messages and getting them onto the UI

Your Response:

When a player clicks an answer, the client sends that choice to the server. The server checks if it's correct or wrong, then creates a message like "Preet locked in the correct answer." It sends that message to all players in the room. Each client receives it and updates the game events panel so everyone sees the result in real time.



Saved: 8/11/2025 11:12:38 PM

Task #3 (0.67 pts.) - Game Area

Progress: 50%

Details:

- Show the question category
- Show the current question
- Show the current answers; each with a button to lock-in that choice
 - Locking-in changes the color of the button and disables all other choices
 - Round start resets this

Part 1:

Progress: 0%

Details:

- Show a few examples of categories/questions and locked-in answers across 3+ clients



Missing Caption



Saved: 8/11/2025 10:09:37 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for syncing the category/questions and locked-in status

Your Response:

When a new round starts, the server picks a category and question, then sends them to all clients inside a payload. On the client side, this payload is received and the game UI updates to show the category, question text, and answer buttons. When a player locks in an answer, the client sends a `PayloadType.ANSWER` back to the server. The server stores that choice, marks the player as "locked in," and broadcasts that status to all other clients. The UI then disables that player's answer buttons, highlights their locked-in choice, and updates the status list so everyone sees who has already locked in.



Saved: 8/11/2025 10:09:37 PM

Section #3: (4 pts.) Project Extra Features

Progress: 68%

Task #1 (2 pts.) - Category Selection

Progress: 87%

Details:

- Categories can be toggled by session creator

Part 1:

Progress: 75%

Details:

- Show the Ready Check screen with the option for the host (3+ clients must be visible) - Show the related code that makes this interactable only for the host
- Show the code related to having the server-side utilize only the chosen categories

```
public void addOrUpdateCategoryClicked() {
    if (ClientCategoryManager.getInstance().isHost()) {
        if (ClientCategoryManager.getInstance().isReadyCheckEnabled()) {
            ClientCategoryManager.getInstance().addOrUpdateCategory(category);
        } else {
            ClientCategoryManager.getInstance().addOrUpdateCategory(category);
        }
    } else {
        ClientCategoryManager.getInstance().addOrUpdateCategory(category);
    }
}

public void removeCategoryClicked() {
    if (ClientCategoryManager.getInstance().isHost()) {
        ClientCategoryManager.getInstance().removeCategory(category);
    } else {
        ClientCategoryManager.getInstance().removeCategory(category);
    }
}

public void toggleReadyCheckClicked() {
    if (ClientCategoryManager.getInstance().isHost()) {
        ClientCategoryManager.getInstance().toggleReadyCheck();
    } else {
        ClientCategoryManager.getInstance().toggleReadyCheck();
    }
}
```

Missing Caption



Saved: 8/11/2025 11:38:08 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code for the host's option to toggle this feature
- Briefly explain the code for handling the selected categories (from the UI to the server-side)

Your Response:

Host Toggle – The UI checks if the current client is the host; if yes, category options are enabled, otherwise disabled. Changes send a CATEGORY_UPDATE payload to the server. This ensures only the host can modify session categories.

Selected Categories – When confirmed, the client sends the chosen categories to the server. The server saves them, uses them for questions, and broadcasts them to all clients. This keeps all players in sync with the host's settings.



Saved: 8/11/2025 11:38:08 PM

☰ Task #2 (2 pts.) - Add New Questions

Progress: 50%

Details:

- Users can create new questions
 - Question text
 - Category
 - 2-4 answers
 - Correct answer
 - Gets saved server-side in a file
 - Can be done outside of an active session (i.e., ready check)

🖼️ Part 1:

Progress: 0%

Details:

- Show a few variations of the screen as new questions are added
- Show the updated/generated file when a question is created
- Show an example of the new question in play



Missing Caption



Saved: 8/11/2025 10:12:06 PM

☰ Part 2:

Progress: 100%

Details:

- Briefly explain the code for this feature from the UI to updating the server-side file

Your Response:

The UI collects the question text, category, answers, and correct answer, then sends them to the server in a NEW_QUESTION payload when the user clicks Save. The server receives this payload, validates the data, and appends it to the questions file stored on the server. Finally, the updated file is saved so future sessions can use the new question. and it can appear in play during the

next game.



Saved: 8/11/2025 10:12:06 PM

Section #4: (2 pts.) Project General Requirements

Progress: 50%

≡ Task #1 (1 pt.) - Away Status

Progress: 50%

Details:

- Clients can mark themselves away and be skipped in turn flow but still part of the game
- The status should be visible to all participants
- A message should be relayed to the Game Events Panel (i.e., Bob is away or Bob is no longer away)
- The user list should have a visual representation (i.e., grayed out or similar)

🖼 Part 1:

Progress: 0%

Details:

- Show the UI button to toggle away
- Show the related code flow from UI to server-side back to UI for showing the status
- Show the related code flow for sending the message to Game Events Panel
- Show various examples across 3+ clients of away status (including Game Events Panel messages)
- Show the code that ignores an away user from turn/round logic



Missing Caption



Saved: 8/11/2025 10:16:17 PM

≡ Part 2:

Details:

- Briefly explain the code flow for the away action from UI to server-side and back to UI
- Briefly explain how the server-side ignores the user from turn/round logic

Your Response:

Clicking "Away" in the UI sends an AWAY_STATUS payload to the server with the player's ID and status. The server updates the status, broadcasts it to all clients, and sends a game event message. Clients gray out away players and show the update in the Game Events Panel.

Ignoring in turn/round logic: The server's turn loop skips any player marked as away, ensuring only active players take turns.



Saved: 8/11/2025 10:16:17 PM

☰ Task #2 (1 pt.) - Spectators

Progress: 50%

Details:

- Spectators are users who didn't mark themselves ready
 - Optionally you can include a toggle on the Ready Check page
- They can see all chat but are ignored from turn/round actions and can't send messages
- Spectators will have a visual representation in the user list to distinguish them from other players
- A message should be relayed to the Game Events Panel that a spectator joined (i.e., during an in-progress session)

Part 1:

Progress: 0%

Details:

- Show the UI indicator of a spectator (visual and message)
- Show the related code flow from UI to server-side back to UI for showing the status
- Show the related code flow for sending the message to Game Events Panel
- Show various examples across 3+ clients of spectator status (including Game Events Panel messages)
- Show the code that ignores a spectator from turn/round logic
- Show the code that prevents spectators from sending messages (server-side)
- Show the spectator's view of the session
- Show the code related to the spectator seeing the session data (including things participants won't see)



Missing Caption

 Saved: 8/11/2025 10:17:59 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for the spectator logic from server-side and to UI
- Briefly explain how the server-side ignores the user from turn/round logic
- Briefly explain the logic that prevents spectators from sending a message
- Briefly explain the logic that shares extra details to the spectator (information normal participants won't see such as the correct answer)

Your Response:

Spectator logic (server to UI): When a player is flagged as a spectator, the server stores this status in their client data and sends an update to all clients, which refresh the UI to display a spectator icon or style.

Ignoring spectators in turns/rounds: The server's turn/round selection logic skips over any client marked as a spectator when determining whose turn it is.

Preventing spectators from sending messages: Before processing chat input, the server checks if the client is a spectator; if so, it blocks the message from being broadcast.

Sharing extra details with spectators: When sending question/answer data, the server includes the correct answer in packets sent only to spectators, letting them see information hidden from active players.

 Saved: 8/11/2025 10:17:59 PM

Section #5: (1 pt.) Misc

Progress: 66%

≡ Task #1 (0.33 pts.) - Github Details

Progress: 0%

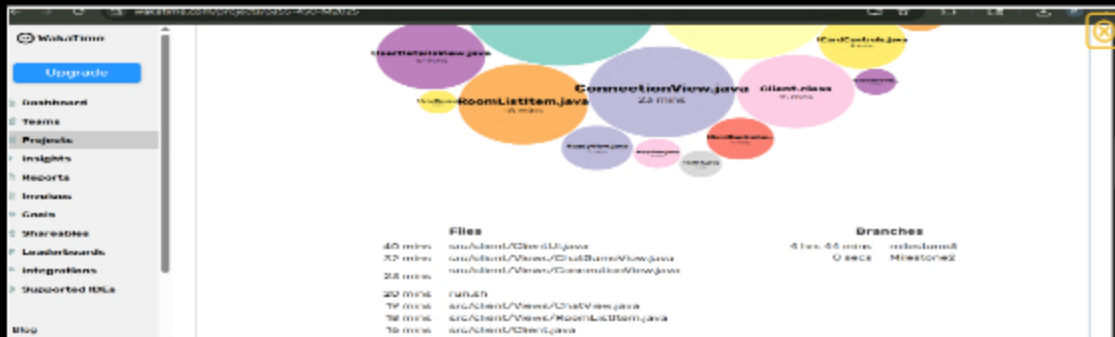
— Task Collapsed —

Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



Spent More hours, taking ss earlier to get the task done



Saved: 8/11/2025 10:33:14 PM

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I learned how different parts of a multiplayer game system communicate — from capturing input in the UI, sending it to the server, processing it there, and then updating all clients so everyone sees consistent data. I also got a better understanding of how to handle special states like ready, away, and spectator through status flags and UI indicators. Finally, I learned how to maintain synchronized ordering and game flow logic across multiple clients while ensuring certain players are skipped or given extra information based on their role.

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of the assignment was implementing the basic UI elements like buttons, labels, and panels for features such as the ready check or away toggle. These were straightforward because they mainly involved standard Swing components and simple event listeners. Once the UI was in place, wiring the button clicks to call the right client methods felt intuitive and didn't require much debugging.

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of the assignment was making sure all the real-time updates between the server and multiple clients stayed perfectly in sync. Handling states like ready status, locked-in answers, or away mode required careful coordination so every client saw the same information at the same time. Debugging these issues was tricky because even small timing or logic mistakes on the server could cause inconsistent displays across clients.